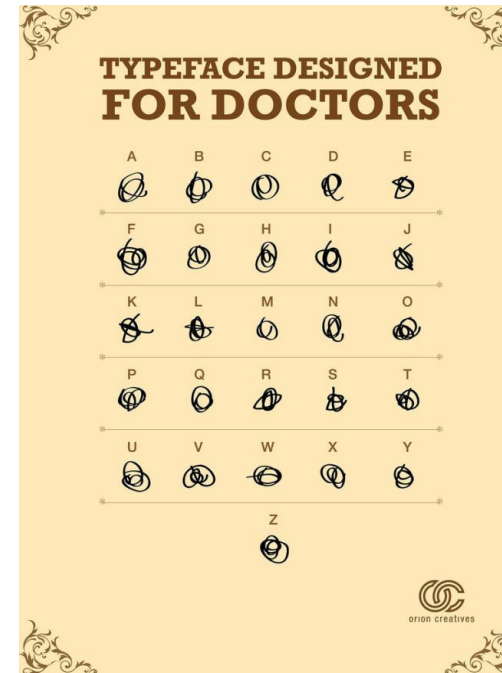


- ▶ Face / speech recognition
- ▶ Stock price prediction
- ▶ Handwriting recognition
- ▶ Play games

Instead of writing program by hand, give computer a lot of examples / **data** with the **output**, so computer can **learn** to improve

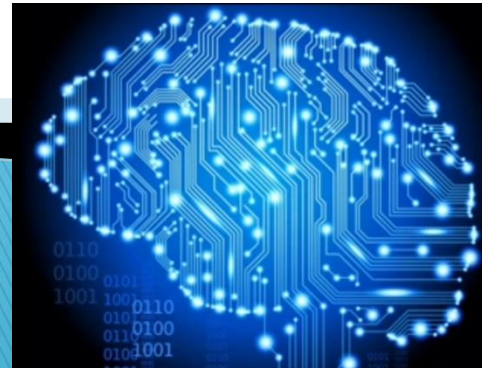


COMP1008 - Fundamentals of AI

Machine Learning – Supervised Learning

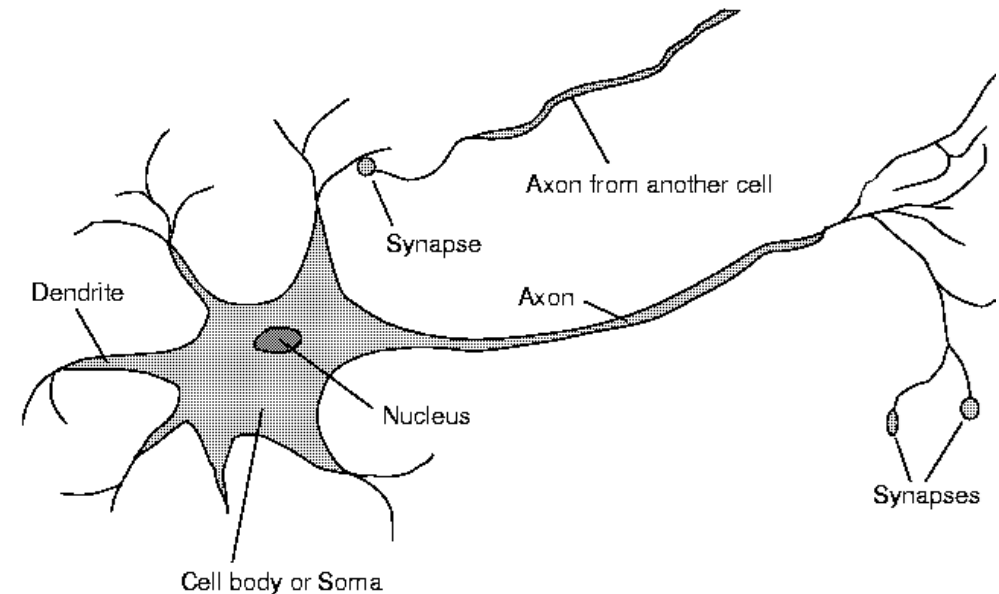
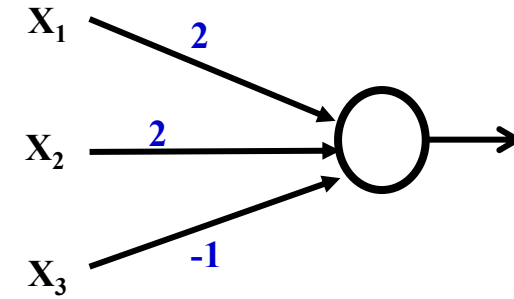
Artificial Neural Networks (ANN)

Professor Rong Qu
rong.qu@nottingham.ac.uk



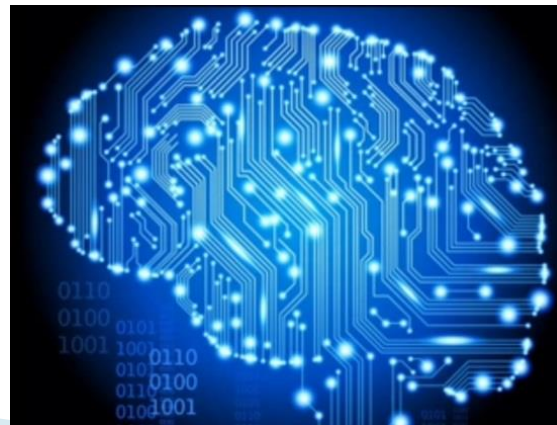
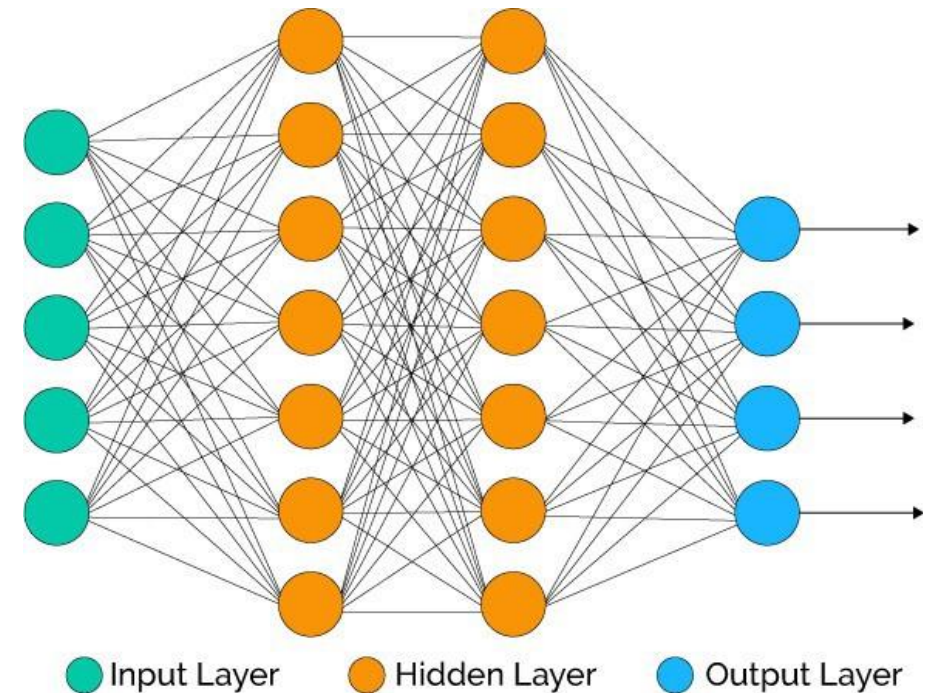
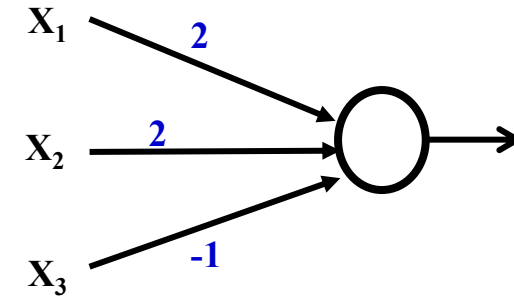
The First Neural Networks

- ▶ More precisely: **Artificial Neural Networks**
 - Simulate, in the computer, neural networks in the brain
- ▶ **McCulloch & Pitts** (1943): the 1st neural networks
 - Combine simple inputs, the idea of a **threshold**
 - Many of their ideas still used today
- ▶ **Hebb** (1949): the **first learning rule**
 - McCulloch & Pitts network: fixed weights
- ▶ During the 50's and 60's
 - **Perceptron**: great excitement
 - Converge to correct **weights**: learning → thinking



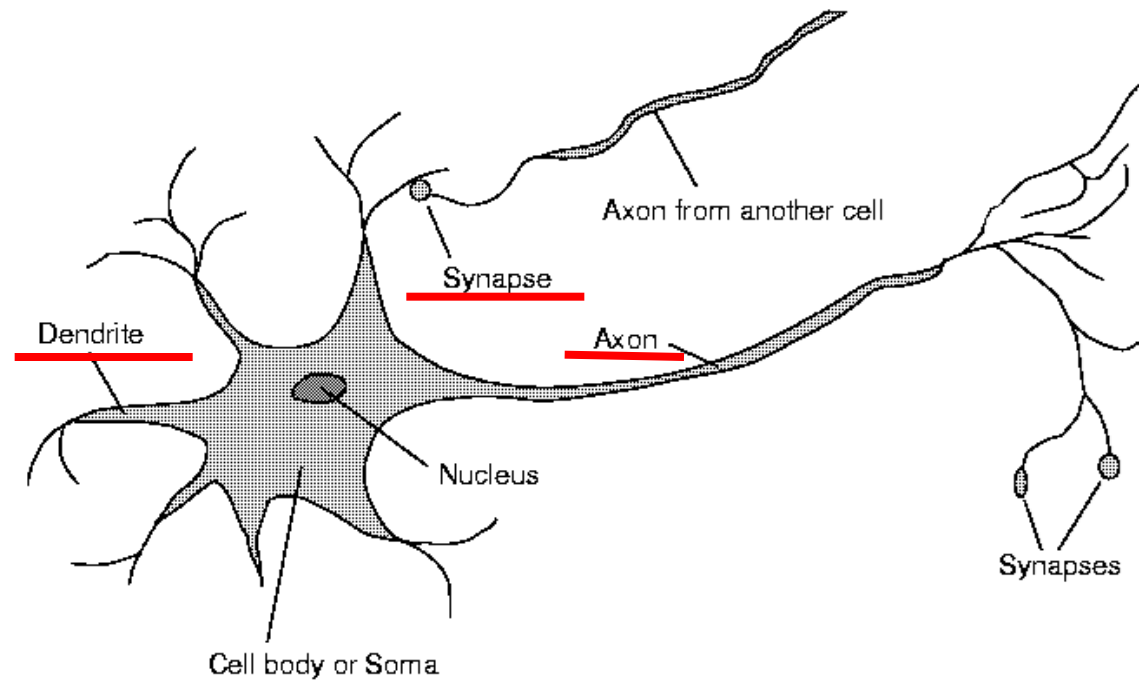
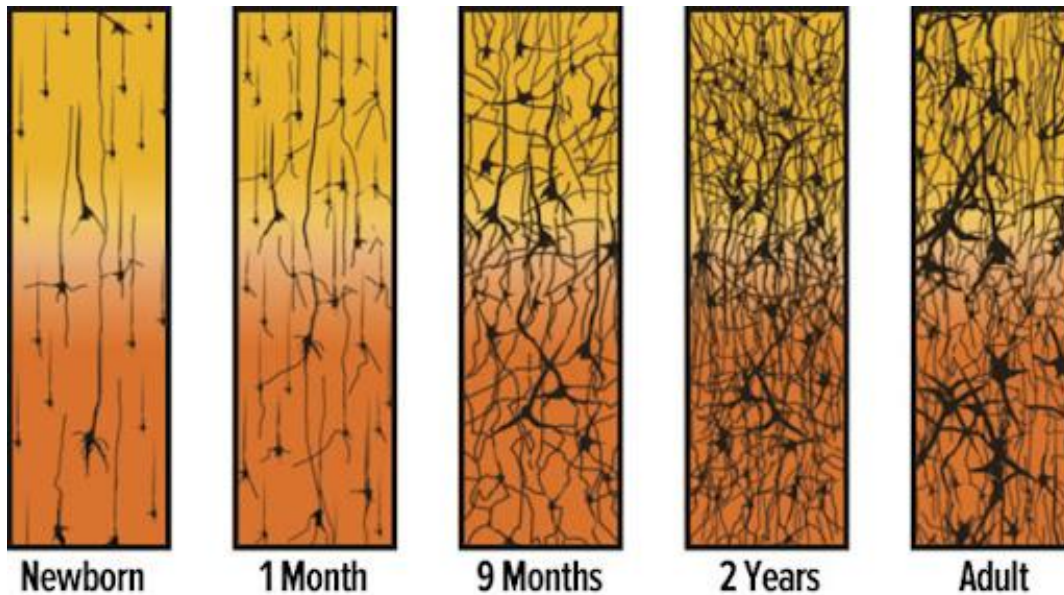
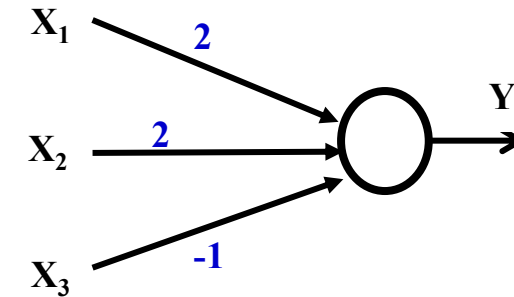
The First Neural Networks

- ▶ 1969: death / winter of ANN research
 - Minsky & Papert
 - Perceptron can't learn certain type of important functions (**non-linear separable**)
 - Declined for about 15 years – “**AI winter**”
- ▶ In the mid 80's: interest revived
 - Parker (1985) and LeCun (1986) independently discovered **multi-layer networks** to solve problem of **non-linear separable**



The First Neural Networks: **Model**

- ▶ Sum of inputs $\geq$ **threshold**, the neuron "fires"
- ▶ Long-term firing patterns - basic **learning**



The First Neural Networks: Model

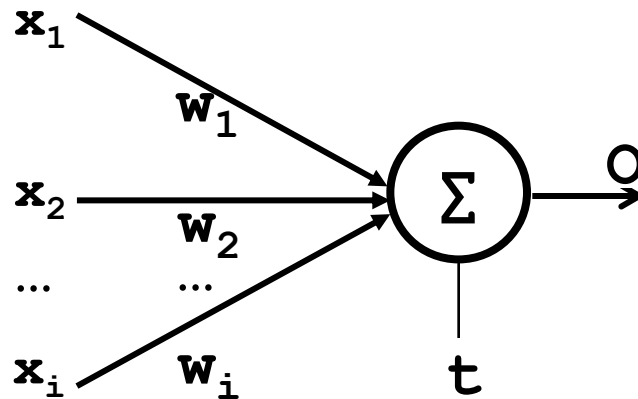
A **processing element** - (neuron)

A set of **inputs** - (dendrites)

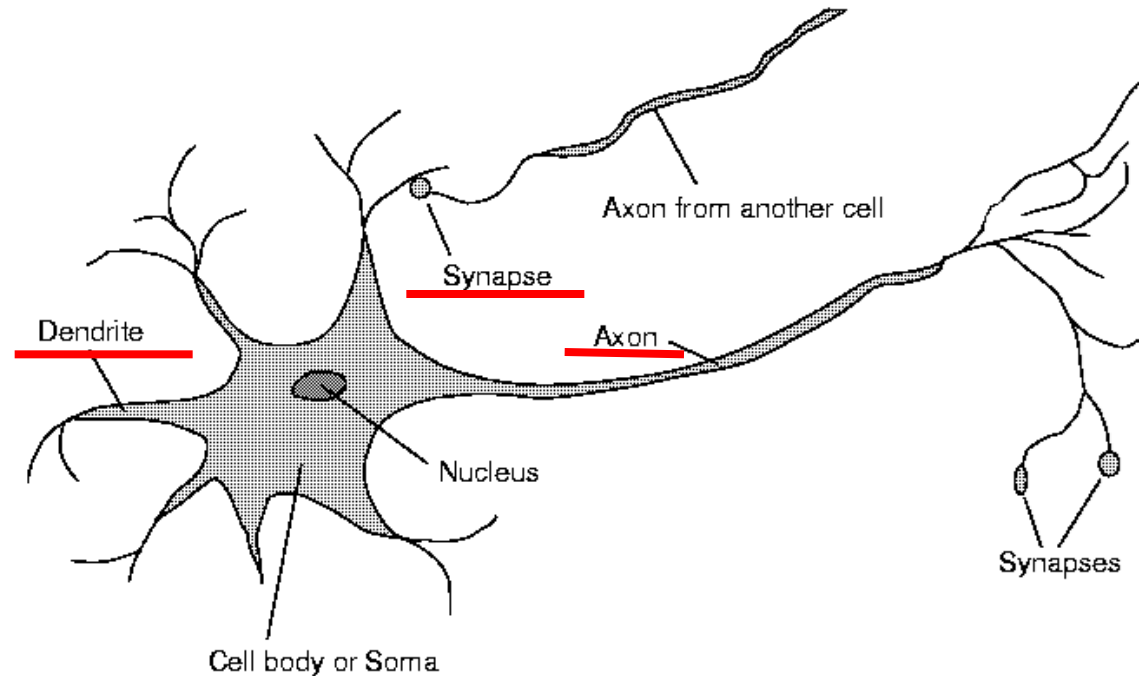
A single **output** - (axon)

A set of **weights** – (synapses)

Perceptron



A computational point of view: fundamental processing unit of a brain: a **neuron**



The First Neural Networks: Model

Neurons connected by directed, weighted paths

Positive weight: **excitatory**, otherwise **inhibitory**

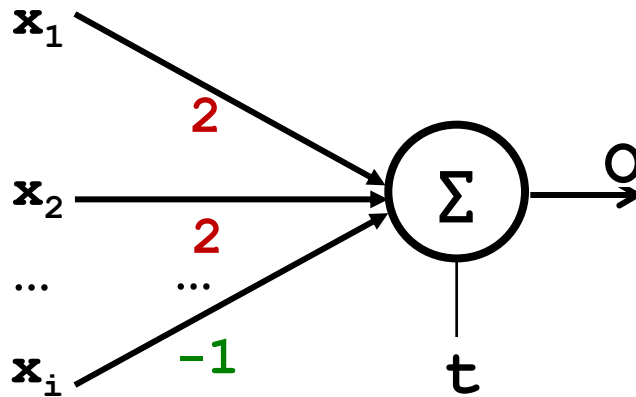
x_1 and x_2 **encourage** the neuron to fire

x_i **prevents** the neuron from firing

Each neuron has a fixed **threshold t**

Weighted sum input to the neuron $\geq t$

then the neuron **fires**

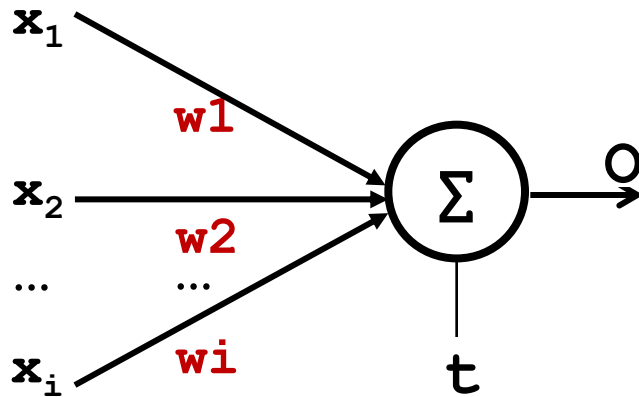


$$O = x_1 * 2 + x_2 * 2 + x_3 * -1$$

Weighted sum

The First Neural Networks: Model

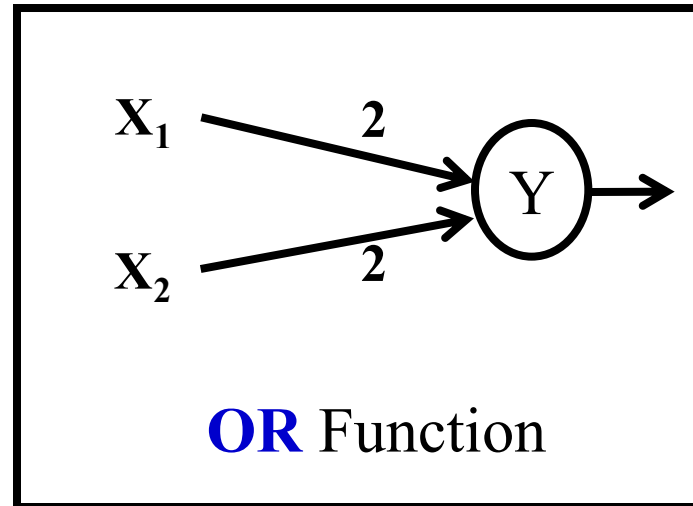
McCulloch-Pitts models can model logic functions
Let's look at 4 logic functions



$$O = x_1 * w1 + x_2 * w2 + \dots x_i * wi$$

Weighted sum

The First Neural Networks: **Modeling OR**



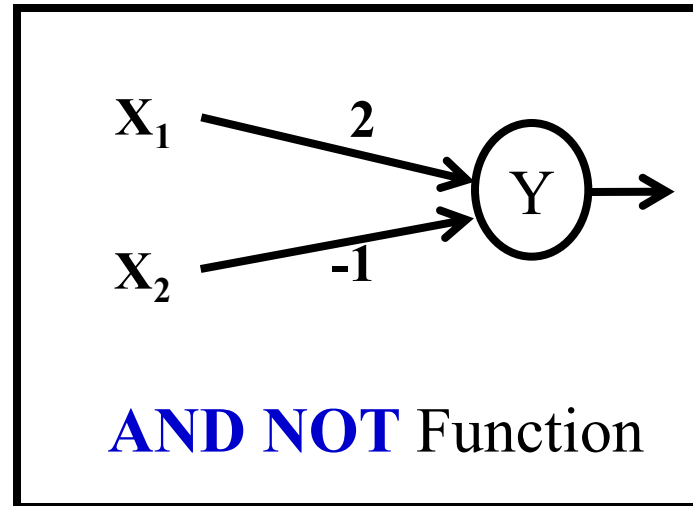
$$\text{Threshold}(Y) = 2$$

Assumption: fixed topology and fixed threshold

OR		
X1	X2	Y
1	1	1
1	0	1
0	1	1
0	0	0

$$Y = x_1 * w1 + x_2 * w2$$

The First Neural Networks: **Modeling AND NOT**



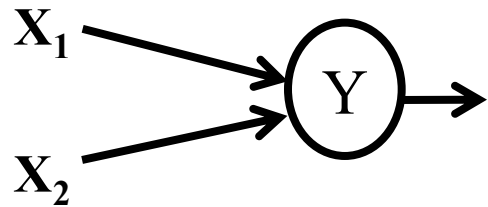
$$\text{Threshold}(Y) = 2$$

AND NOT		
X1	X2	Y
1	1	0
1	0	1
0	1	0
0	0	0

$$Y = x_1 * \textcolor{red}{w1} + x_2 * \textcolor{red}{w2}$$

The First Neural Networks: Modeling XOR

$$X_1 \text{ XOR } X_2 = (X_1 \text{ AND NOT } X_2) \text{ OR } (X_2 \text{ AND NOT } X_1)$$

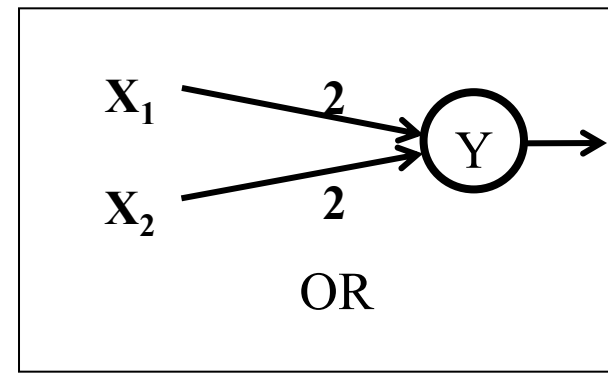
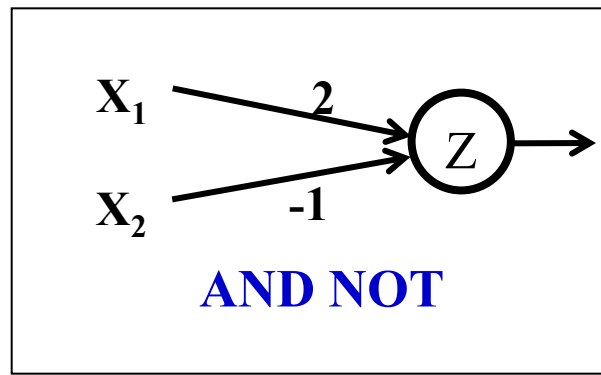
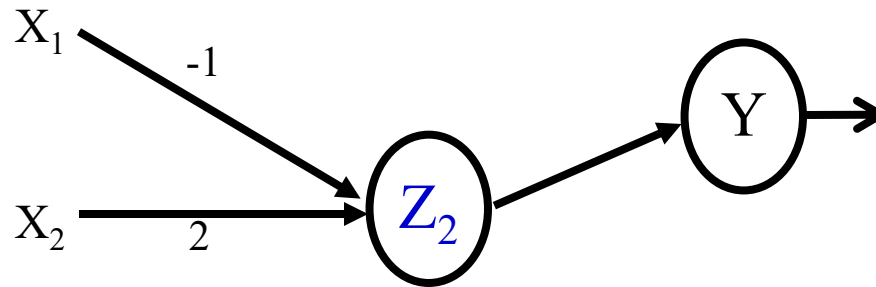


XOR				
X1	X2	Z1	Z2	Y
1	1	0	0	0
1	0	1	0	1
0	1	0	1	1
0	0	0	0	0

AND NOT		
X1	X2	Y
1	1	0
1	0	1
0	1	0
0	0	0

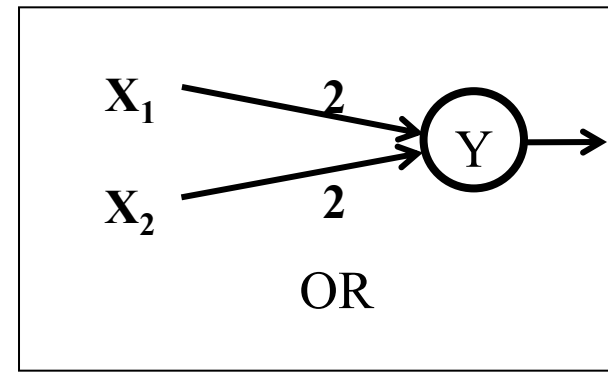
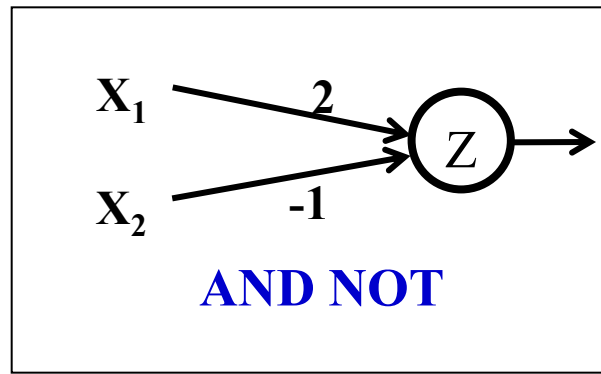
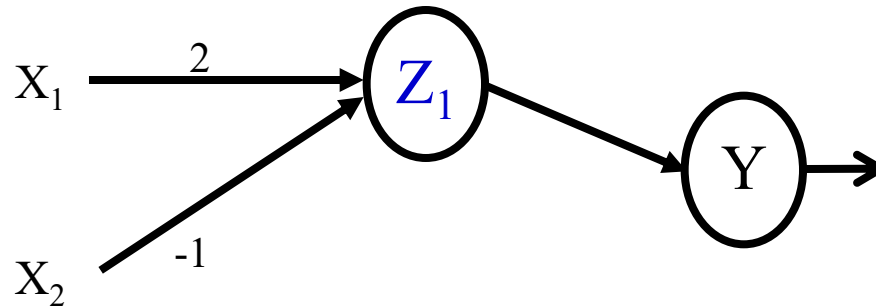
The First Neural Networks: Modeling XOR

$$X_1 \text{ XOR } X_2 = (X_1 \text{ AND NOT } X_2) \text{ OR } (X_2 \text{ AND NOT } X_1)$$



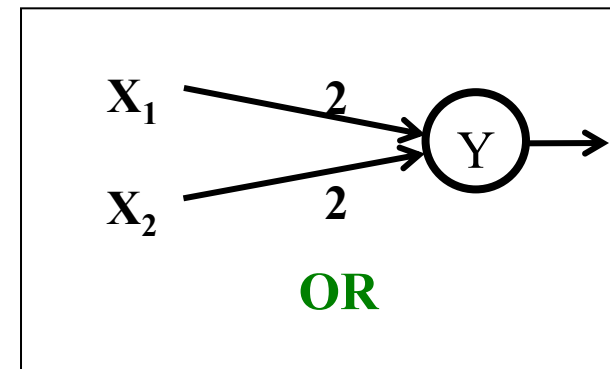
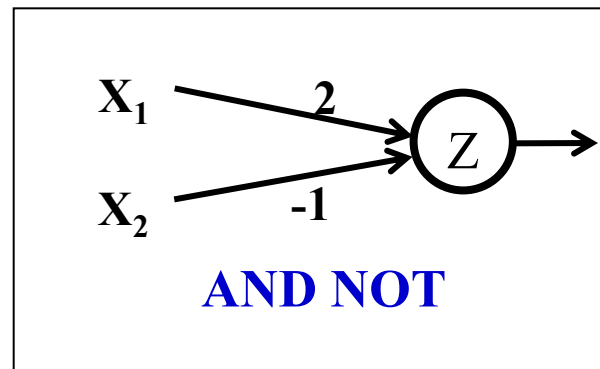
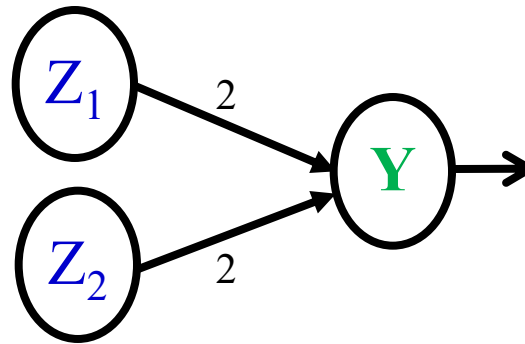
The First Neural Networks: Modeling XOR

$$X_1 \text{ XOR } X_2 = (X_1 \text{ AND NOT } X_2) \text{ OR } (X_2 \text{ AND NOT } X_1)$$

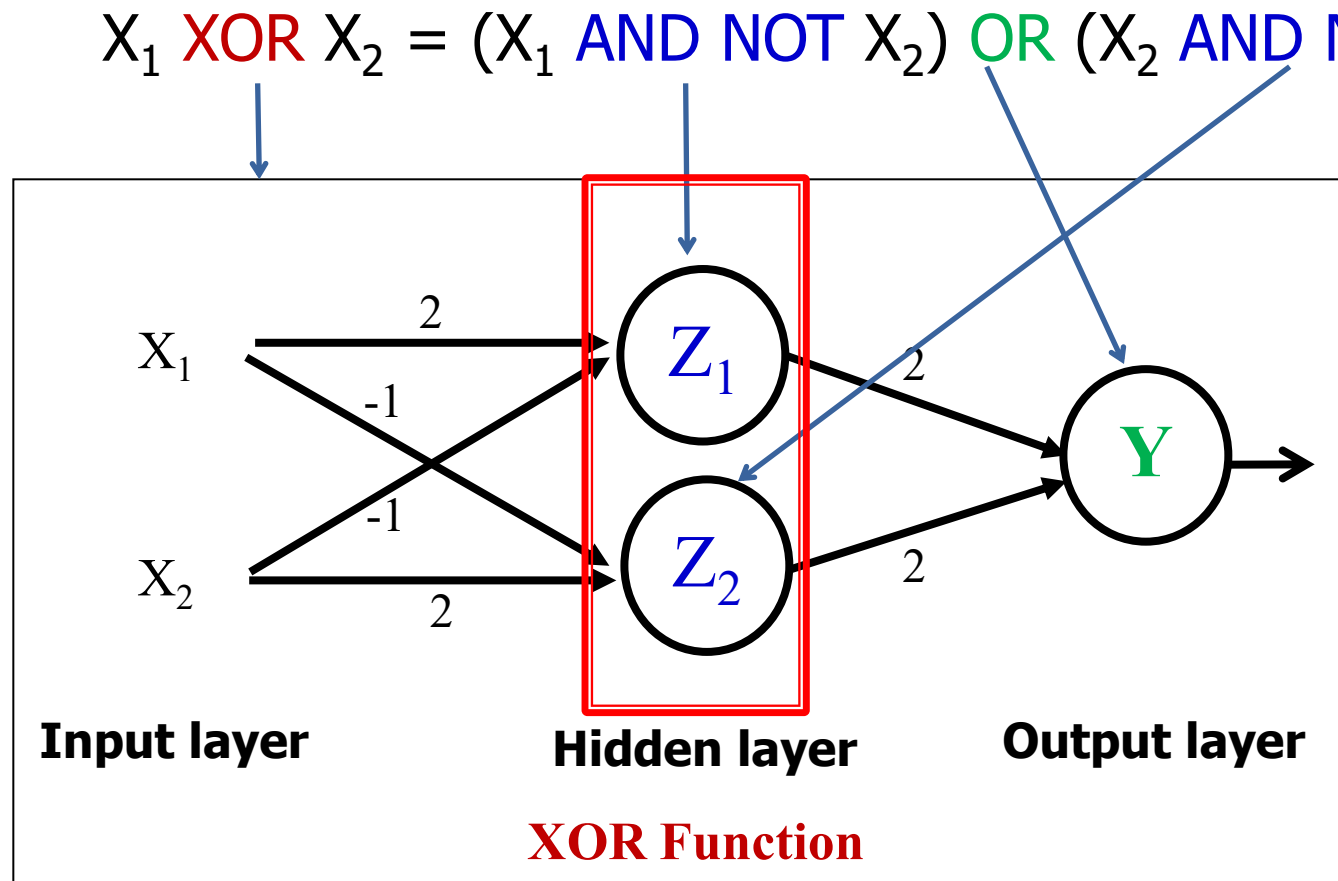


The First Neural Networks: Modeling XOR

$$X_1 \text{ XOR } X_2 = (X_1 \text{ AND NOT } X_2) \text{ OR } (X_2 \text{ AND NOT } X_1)$$

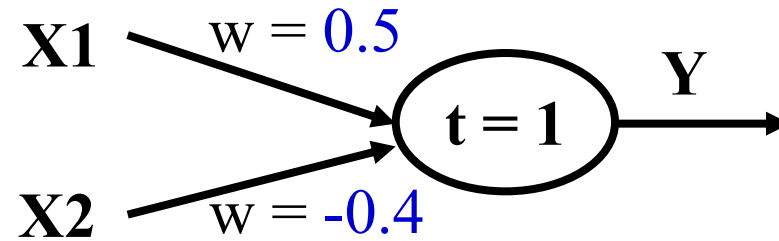


The First Neural Networks: Modeling XOR



Art by Simon Fisher www.simonfishermaritime.com

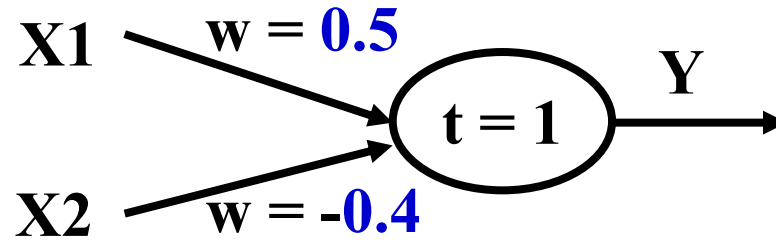
Neural Networks: Training



AND		
X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1

X1	X2	Summation	Output
0	0	$(0 * 0.5) + (0 * -0.4) = 0$	0
0	1		
1	0		
1	1		

Neural Networks: Training



AND			
X1	X2	Y	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0

X1	X2	Summation	Output
0	0	$(0 * 0.5) + (0 * -0.4) = 0$	0
0	1	$(0 * 0.5) + (1 * -0.4) = -0.4$	0
1	0	$(1 * 0.5) + (0 * -0.4) = 0.5$	0
1	1	$(1 * 0.5) + (1 * -0.4) = 0.1$	0

Hebb (1949): the first learning rule

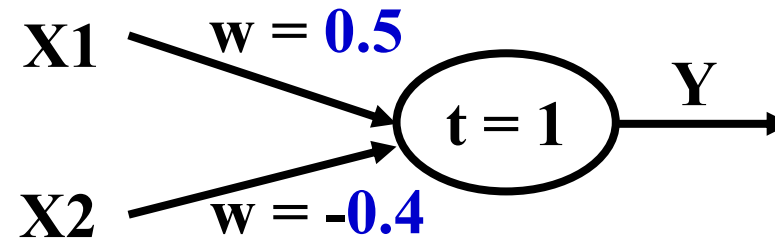
Neural Networks: Training

While epoch produces an error

Check next inputs (pattern) from epoch

AND			
X1	X2	Y	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0

End While



Epoch: The entire training set feed into the neural network.

The AND function: an epoch consists of four sets of inputs (patterns) feed into the network ([0,0], [0,1], [1,0], [1,1])

Neural Networks: Training

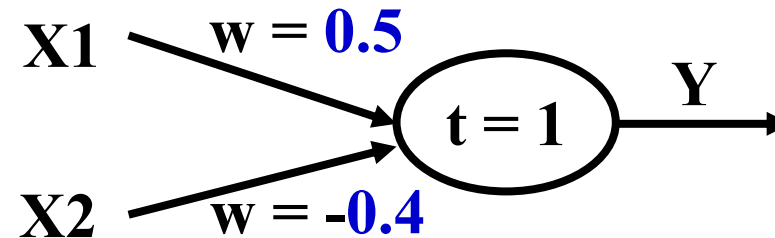
While epoch produces an error

Check next inputs (pattern) from epoch

$$\text{Err} = Y - O$$

AND			
X1	X2	Y	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0

End While



Training Value, Y : The value that we require the network to produce. For example, with [0,0] for the AND function the training value will be 0.
(supervised learning)

Neural Networks: Training

While epoch produces an error

Check next inputs (pattern) from epoch

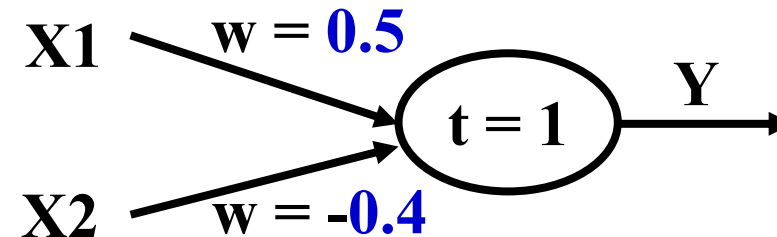
$Err = Y - O$

If $Err \neq 0$ then

End If

End While

AND			
X1	X2	Y	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0



Error, Err : The output by the network O differs from the training value Y.

Neural Networks: Training

While epoch produces an error

Check next inputs (pattern) from epoch

$$\text{Err} = Y - O$$

If $\text{Err} \neq 0$ then

$$w_i = w_i + \text{LR} * X_i * \text{Err}$$

End If

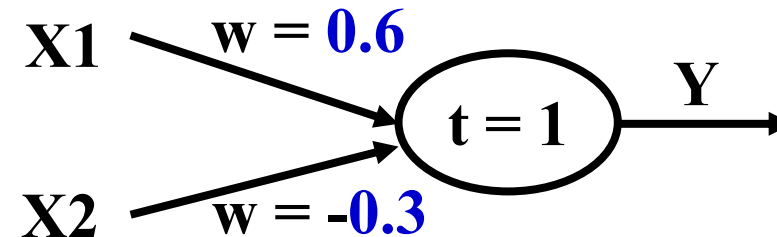
End While

X_i : Inputs to the neuron

w_i : Weight from input X_i to the output

LR: The learning rate: how quickly the network converges. It is set by experimentation, typically 0.1

AND			
X1	X2	Y	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0



What is the problem if the learning rate is set too high, or too low?

Neural Networks: Training

While epoch produces an error

Check next inputs (pattern) from epoch

$$\text{Err} = Y - O$$

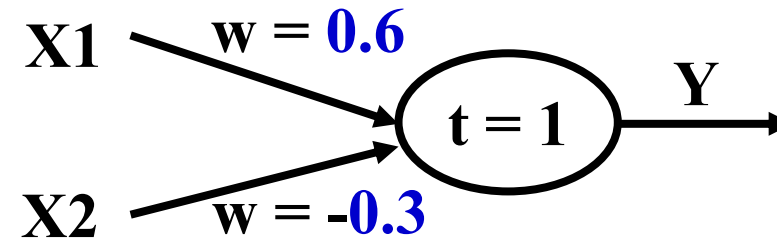
If $\text{Err} \neq 0$ then

$$w_i = w_i + \text{LR} * X_i * \text{Err}$$

End If

End While

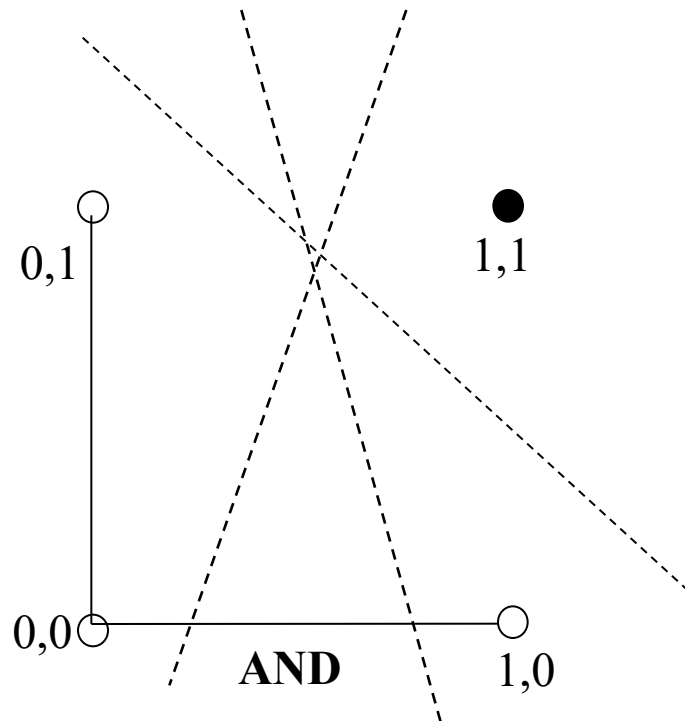
AND			
X1	X2	Y	O
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	0



Performance measure (loss function):

$$\text{Mean Squared Error (MSE)} = [(Y - O)^2] / n$$

Neural Networks: Training



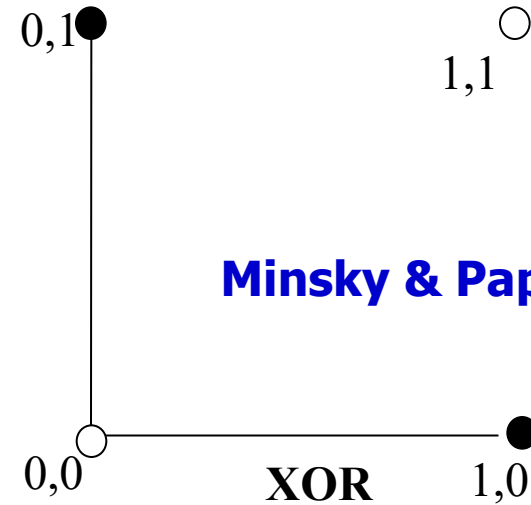
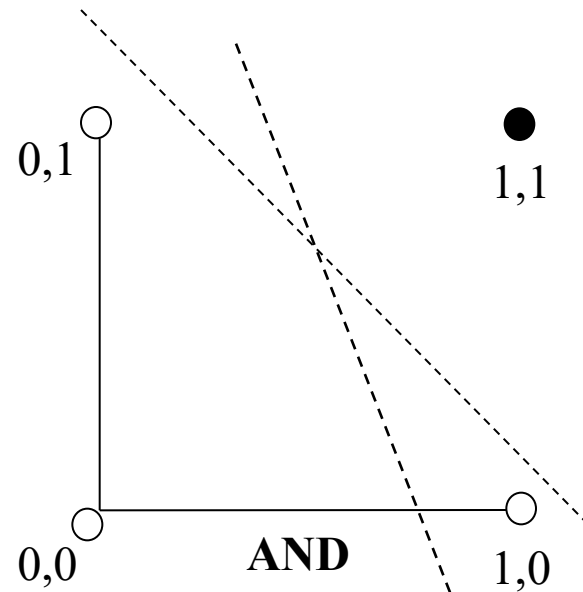
AND		
X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1

$$Y = w0 + x_1 * w1 + x_2 * w2$$

Neural Networks: Linearly Separable

Linearly separable: where output data points (classes) can be separated using a linear boundary (straight line, plane or hyperplane)
Only Linearly Separable functions can be represented by a single layer NN, i.e. perceptron

AND		
X1	X2	Y
0	0	0
0	1	0
1	0	0
1	1	1



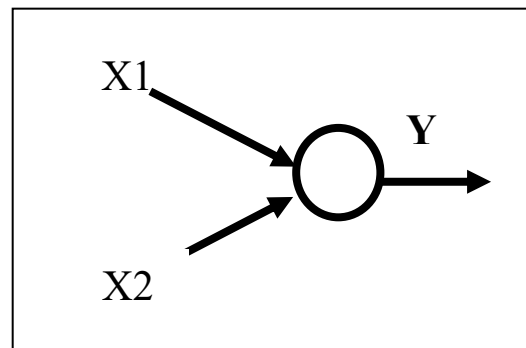
XOR		
X1	X2	Y
0	0	0
0	1	1
1	0	1
1	1	0

Neural Networks: Past Exam Question

The truth table of a function is given in the table. Is it possible for a perceptron to learn the required output?

Explain the reason behind your decision. (4 marks)

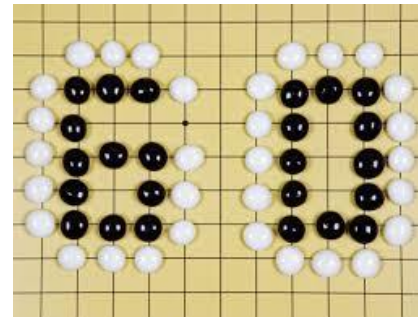
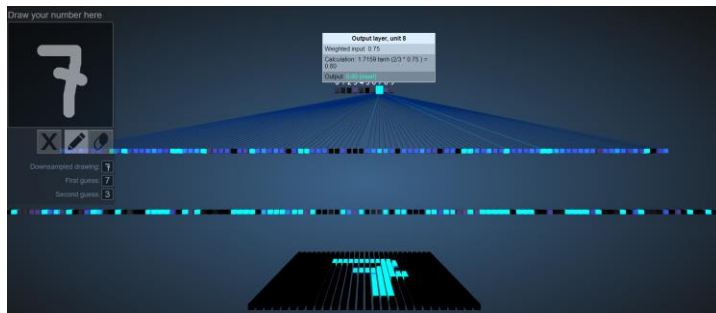
Input X1	0	0	1	1
Input X2	0	1	0	1
Required Output Y	0	1	0	1



Linearly separable?

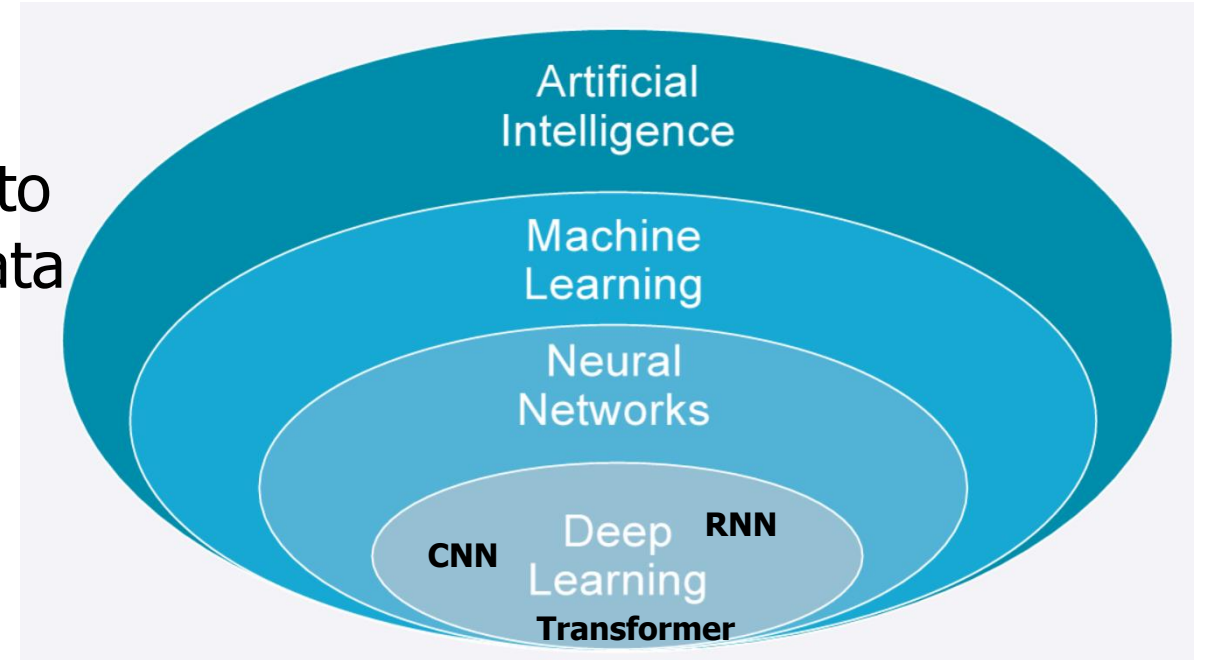
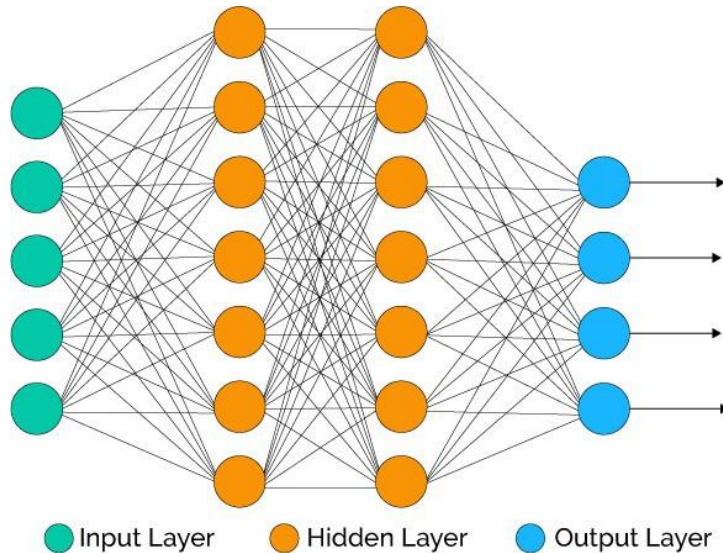
ANN: Pros vs. Cons and Applications

Pros	Cons
Can learn more complicated class boundaries	Hard to implement : trial and error for choosing parameters and network structure
Can be more accurate	Slow training time
Can handle large number of features	Can overfit the data: find patterns in random noise
	Hard to interpret



Neural Networks Extensions: **Deep Learning**

- ▶ Simulate human brain
- ▶ **Multiple layers** of connected neurons to progressively extract features from data




TensorFlow


PyTorch

Neural Networks Extensions: **Deep Learning**

- ▶ The Turing Award 2018, ACM – “Nobel prize of Computing”
 - Dr Yann LeCun, AI Research Director, Facebook; NYU
 - Dr Yoshua Bengio, University of Montreal
 - Dr Geoffrey Hinton, University of Toronto, Google (pre)
 - Godfather of AI / **Deep Learning**



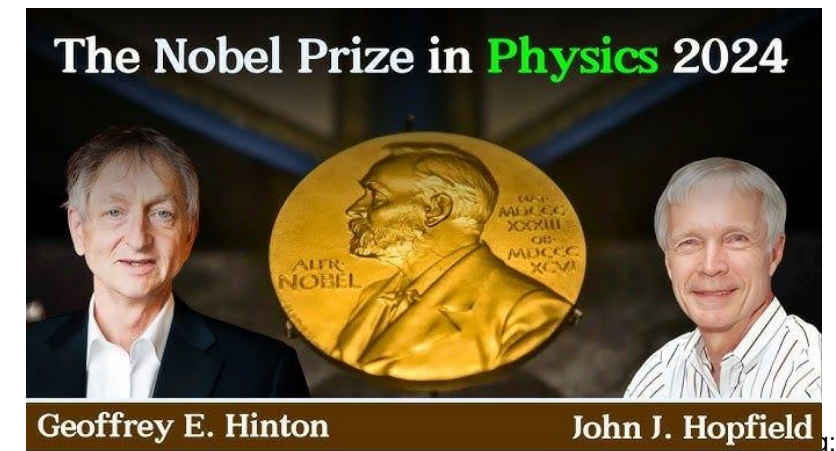
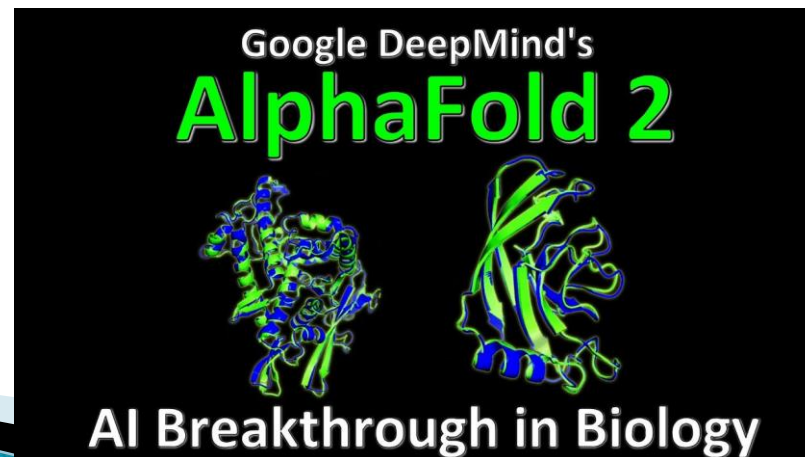
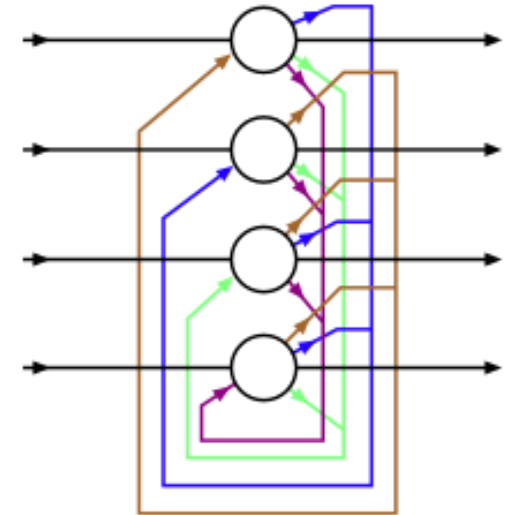
Neural Networks Extensions: **Deep Learning**

► Nobel Prize 2024 Physics

- John J. Hopfield (Princeton University, USA)
- Geoffrey E. Hinton (University of Toronto, Canada)
- **Hopfield Networks**: foundations of machine learning

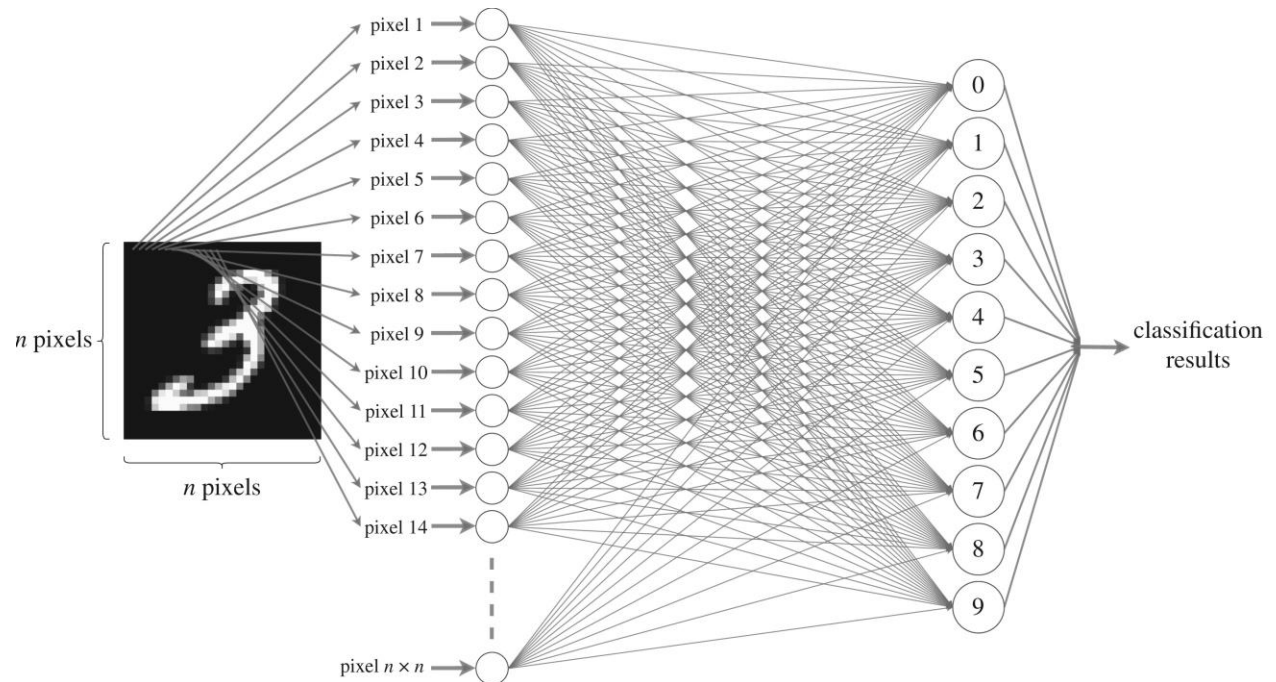
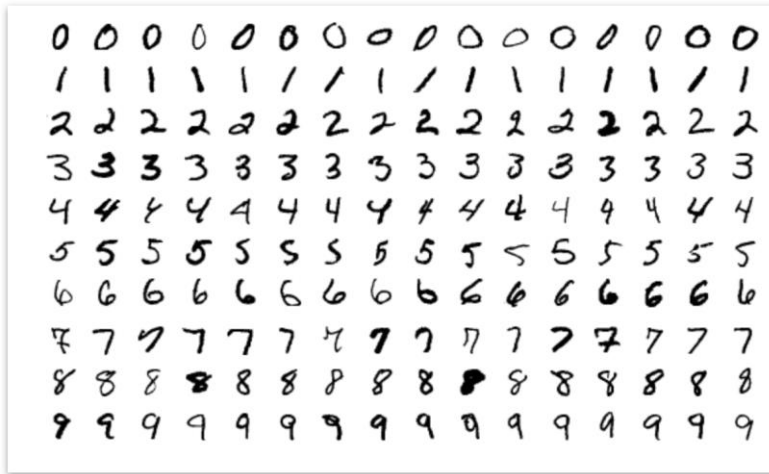
► Nobel Prize 2024 Chemistry

- David Baker, Demis Hassabis, John Jumper
- **AlphaFold2**: Deep learning to predict protein structures from amino acid sequences



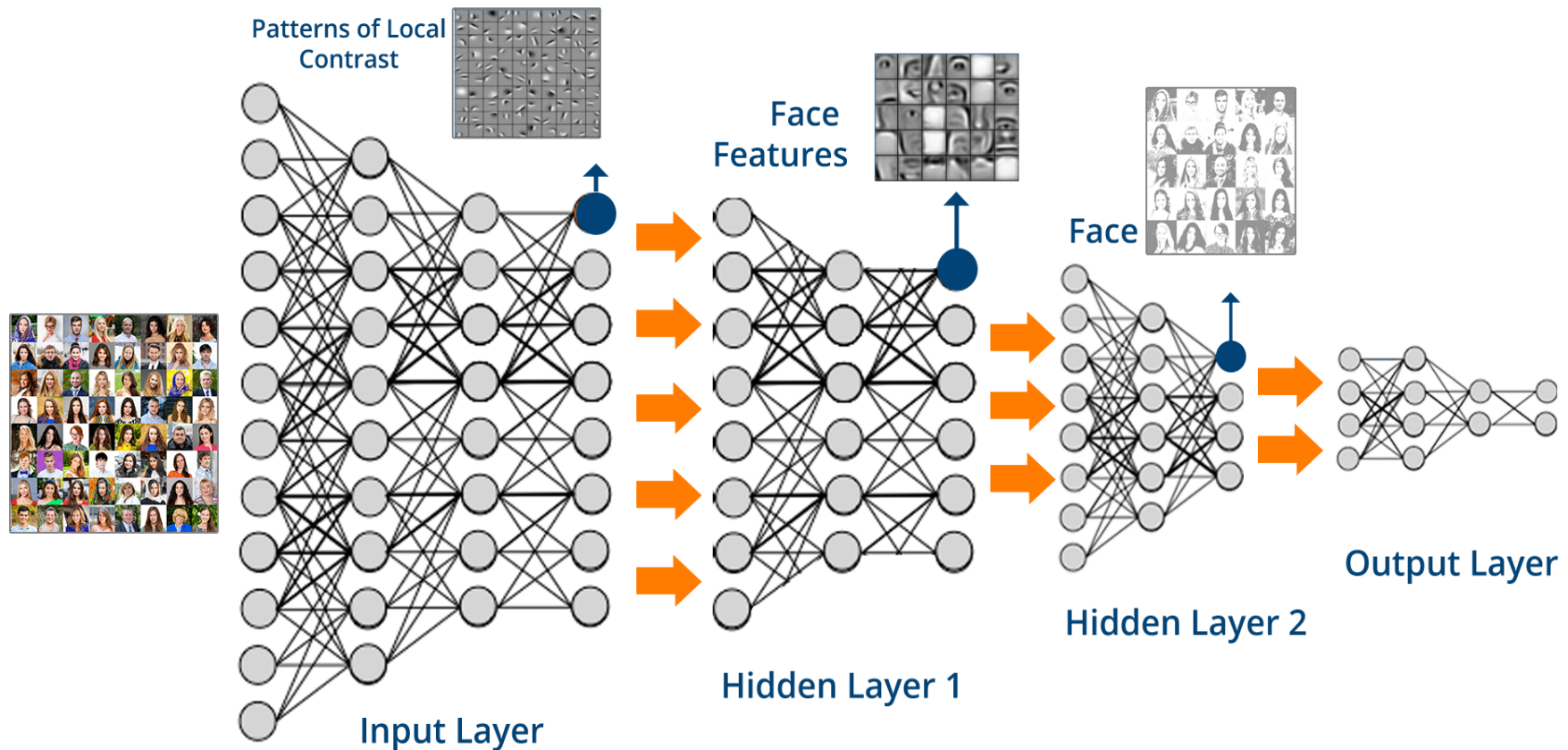
Neural Networks Extensions: CNN

- Convolutional Neural Network (CNN)
 - Deep learning neural network that learns feature engineering by itself via filters



Neural Networks Extensions: CNN

- Convolutional Neural Network (CNN)
 - Deep learning neural network that learns feature engineering by itself via filters

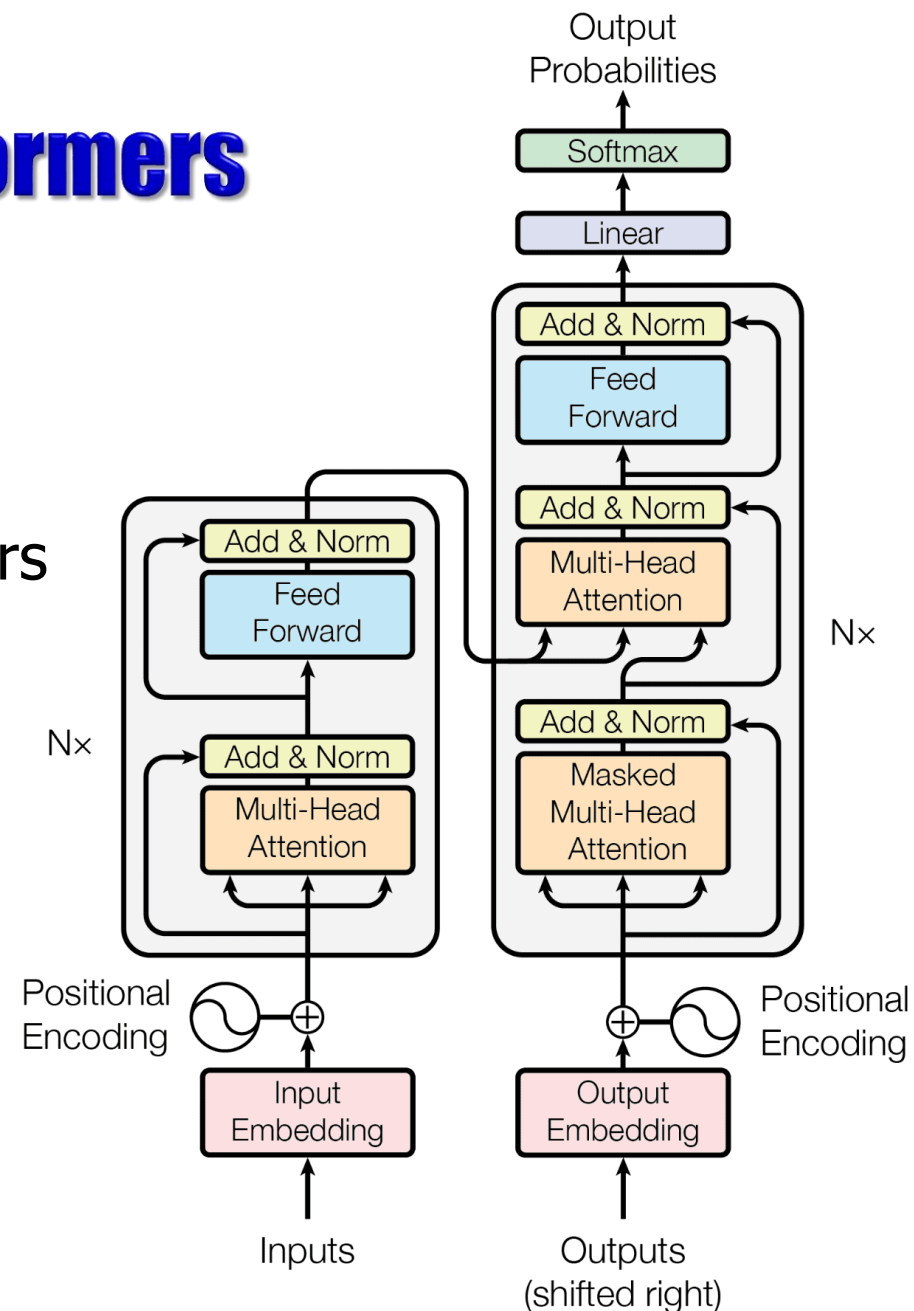


Neural Networks Extensions: Transformers

- ▶ A type of neural networks that transform an input sequence to an output sequence
- ▶ ChatGPT, Gemini, LLMs: build upon transformers
- ▶ COMP1008 Part III

COMP3009 Machine learning

Note: CNN and Transformers are NOT assessed in COMP1008!

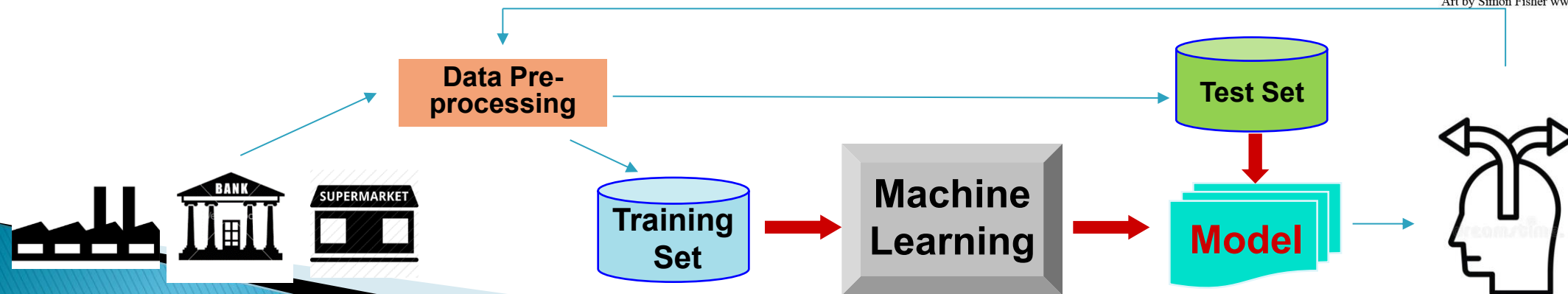


ANN: The Titanic Example

	A	B	C	D	E	F	G	H	I	J	K	L
1	PassengerId	Survived	Pclass	Name	Gender	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
2	1	0	3	Braund, Mr. Owen Harris	male	22	1	0	A/5 21171	7.25		S
3	2	1	1	Cumings, Mrs. John Bradley (Florence	female	38	1	0	PC 17599	71.2833	C85	C
4	3	1	3	Heikkinen, Miss. Laina	female	26	0	0	STON/O2. 3101282	7.925		S
5	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May	female	35	1	0		113803	53.1	C123
6	5	0	3	Allen, Mr. William Henry	male	35	0	0		373450	8.05	S
7	6	0	3	Moran, Mr. James	male		0	0		330877	8.4583	Q
8	7	0	1	McCarthy, Mr. Timothy J	male	54	0	0		17463	51.8625	E46
9	8	0	3	Palsson, Master. Gosta Leonard	male	2	3	1		349909	21.075	S
10	9	1	3	Johnson, Mrs. Oscar W (Elisabeth Vil	female	27	0	2		347742	11.1333	S
11	10	1	2	Nasser, Mrs. Nicholas (Adele Achem	female	14	1	0		237736	30.0708	C
12	11	1	3	Sandstrom, Miss. Marguerite Rut	female	4	1	1	PP 9549		16.7	G6
13	12	1	1	Bonnell, Miss. Elizabeth	female	58	0	0		113783	26.55	C103
14	13	0	3	Saunderscock, Mr. William Henry	male	20	0	0	A/5. 2151		8.05	S
15	14	0	3	Andersson, Mr. Anders Johan	male	39	1	5		347082	31.275	S
16	15	0	3	Vestrom, Miss. Hulda Amanda Adolfi	female	14	0	0		350406	7.8542	S
17	16	1	2	Hewlett, Mrs. (Mary D Kingcome)	female	55	0	0		248706	16	S
18	17	0	3	Rice, Master. Eugene	male	2	4	1		382652	29.125	Q
19	18	1	2	Williams, Mr. Charles Eugene	male		0	0		244373	13	S
20	19	0	3	Vander Planke, Mrs. Julius (Emelia M	female	31	1	0		345763	18	S



Art by Simon Fisher www.simonfisher maritime.com



ANN: Data Preparation

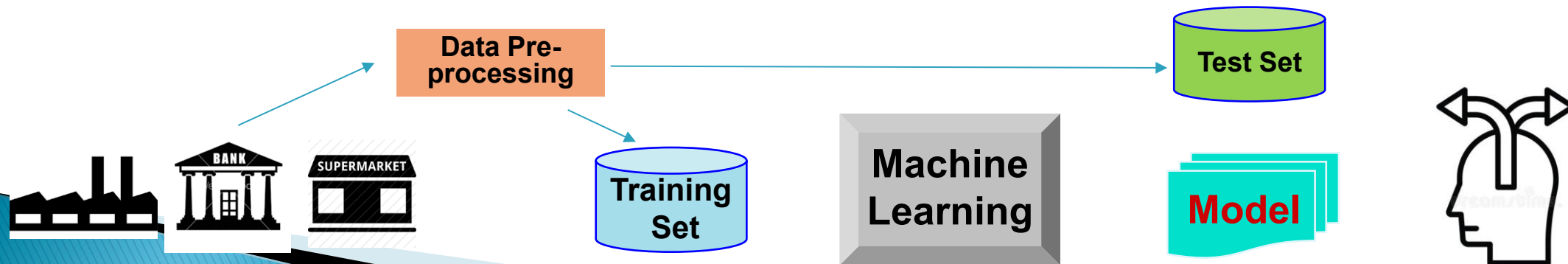
- ▶ Same as that in Decision Tree

missing values; data types

```
from sklearn.model_selection import train_test_split, cross_val_score
```

```
X_train, X_test, y_train, y_test = train_test_split(df_X, df_y, test_size=0.3)
```

Features	Labels
x_train ...	y_train
x_test ...	y_test



ANN: Training

```
# import the sklearn multi-layer perceptron package
from sklearn.neural_network import MLPClassifier
mlp = MLPClassifier(hidden_layer_sizes=(3,6), max_iter=3000)
mlp.fit(X_train, y_train)
```

Features	Labels
x_train ...	y_train
x_test ...	y_test

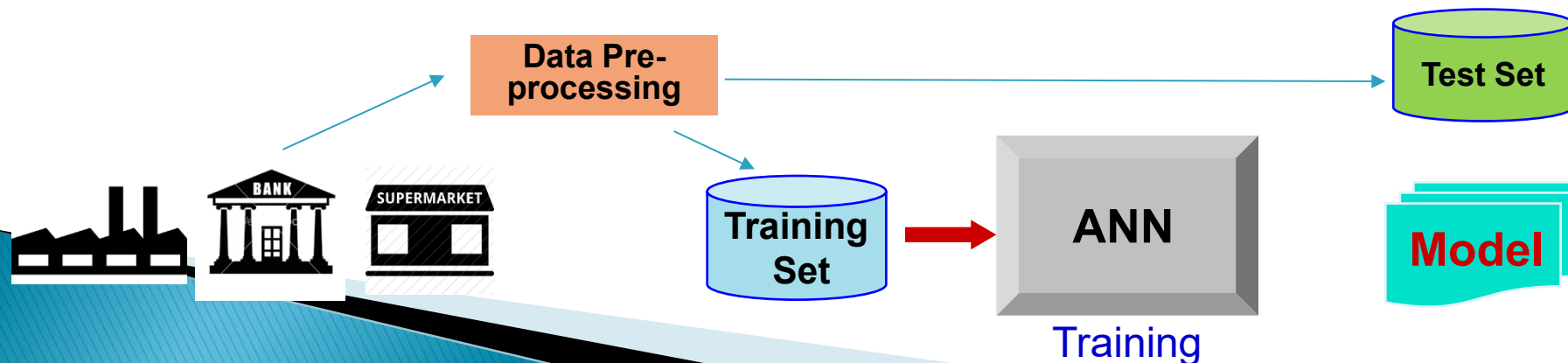
y_train

Survived
0
1
1
1
0



X_train

	pclass	sex	age	sibsp	parch	ticket	fare	cabin	embarked
0	1.0	female	29.0000	0.0	0.0	24160	211.3375	B5	S
1	1.0	male	0.9167	1.0	2.0	113781	151.5500	C22 C26	S
2	1.0	female	2.0000	1.0	2.0	113781	151.5500	C22 C26	S
3	1.0	male	30.0000	1.0	2.0	113781	151.5500	C22 C26	S
4	1.0	female	25.0000	1.0	2.0	113781	151.5500	C22 C26	S
...	...	...	...	...	...	...	...	...	...



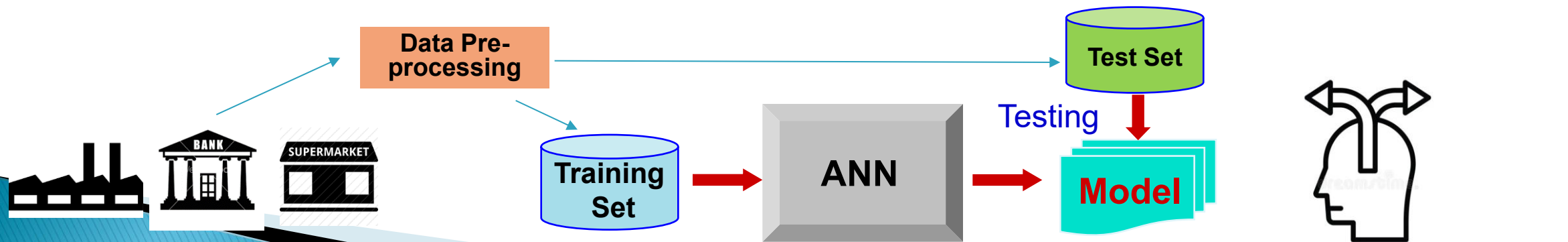
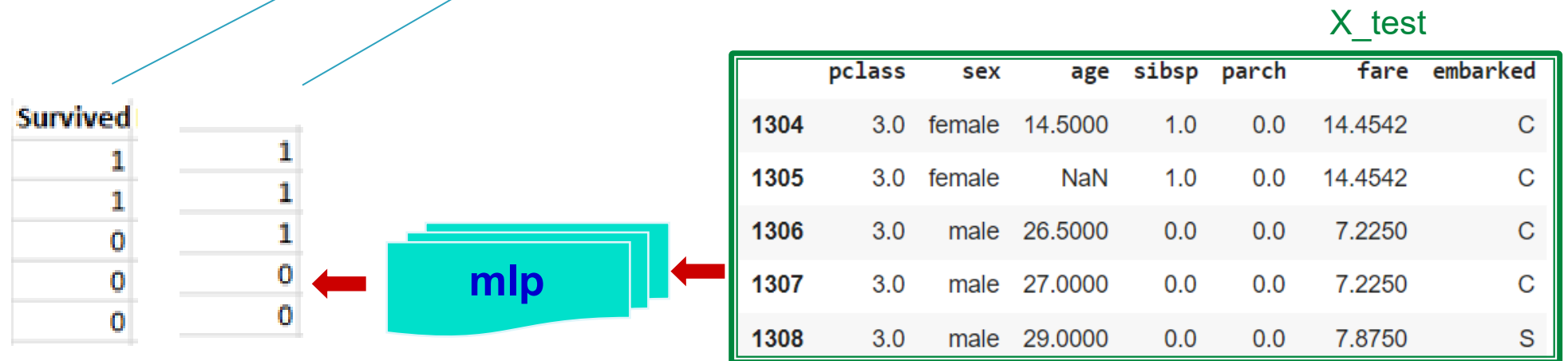
ANN: Testing

```
from sklearn import metrics
```

```
y_pred = mlp.predict(x_test)
```

```
print(metrics.accuracy_score(y_test, y_pred))
```

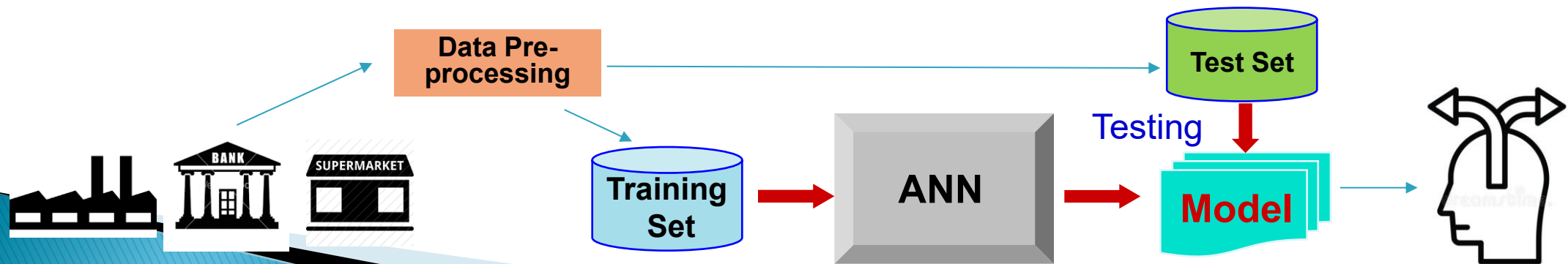
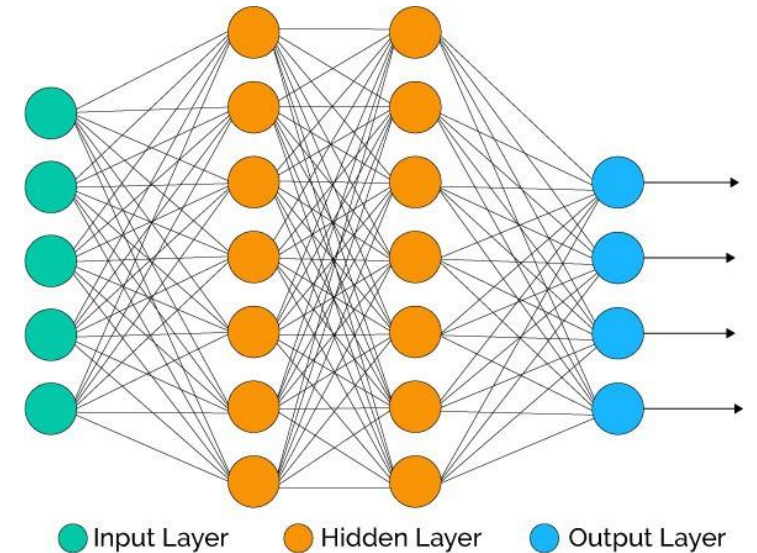
```
from sklearn.model_selection import cross_val_score  
acc_ann = cross_val_score(mlp1, data, target, cv=5, scoring =  
    'accuracy').mean()
```



ANN: Interpretation

- ▶ Trained neural network
 - Difficult to understand
 - Combinations of a series of linear regression

```
print("Number of outputs:", mlp.n_outputs_)  
print("Weights between input and first hidden layer:", mlp.coefs_[0])  
print("Weights between last hidden layer and output layer:", mlp.coefs_[-1])
```

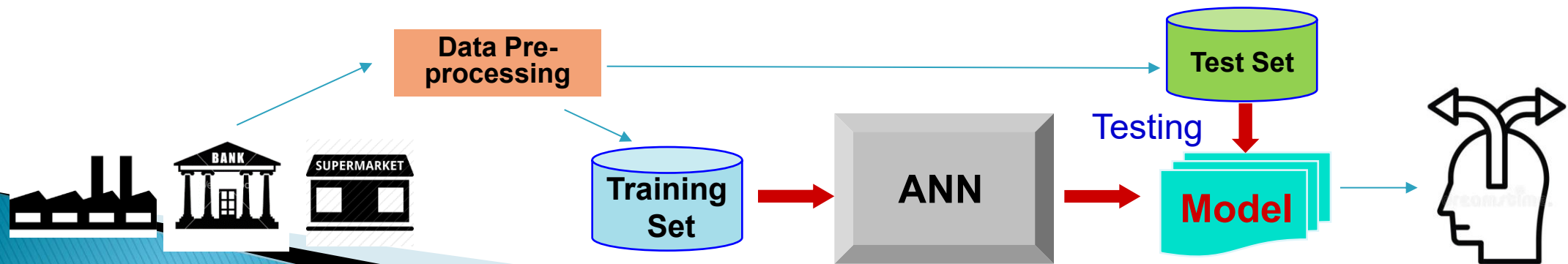
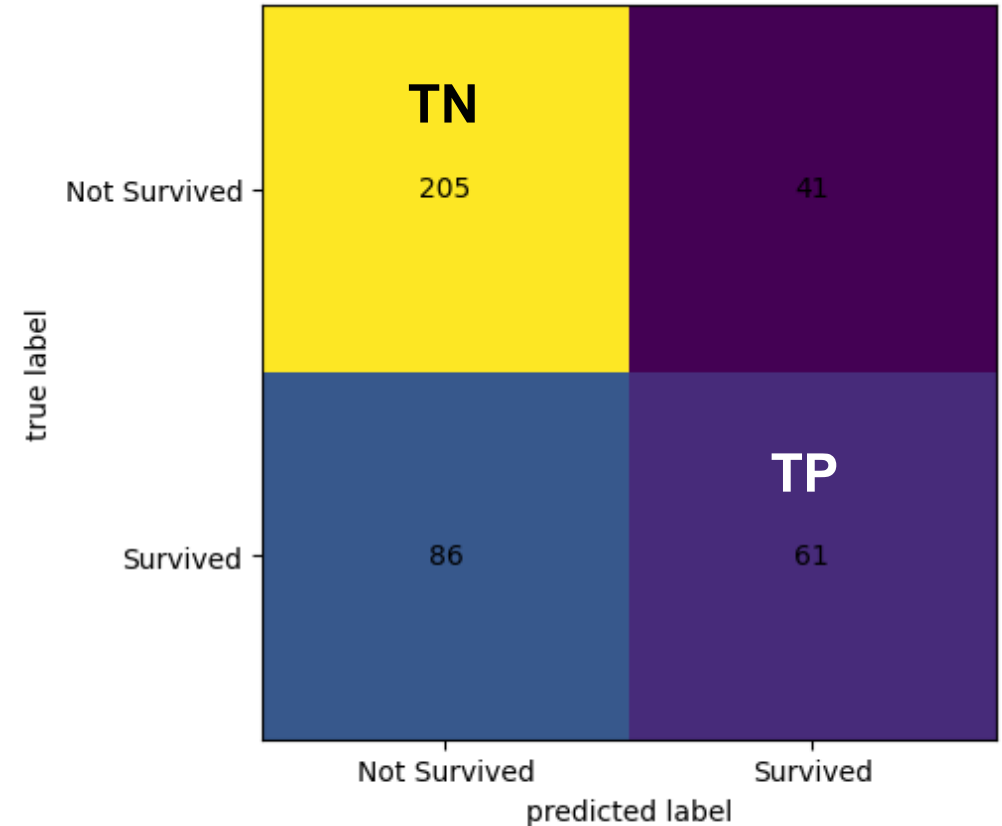


ANN: Interpretation

► Confusion matrix

- performance evaluation, accuracy in details
- true positives (TP), true negatives (TN)
- false positives (FP), and false negatives (FN)

```
from sklearn.metrics import confusion_matrix  
cm_dt = confusion_matrix(y_test, y_predict)  
cm_dt
```



ANN: Classification vs. Regression

- ▶ ANN can solve both regression and classification tasks!
 - ▶ **MLPRegressor** for regression
 - ▶ **MLPClassifier** for classification tasks
- ▶ Note: ANN weights can't reflect the importance of the input features

```
from sklearn.neural_network import MLPRegressor
```

```
# build an ANN regression model
```

```
mlp = MLPRegressor(hidden_layer_sizes=(10,20), max_iter=8000, random_state=1)
```

```
# train the model with the training data
```

```
mlp.fit(x_train, y_train)
```

